

Day 4 — Learning Content

Shift Registers, Function Blocks & PID — full teaching script

1. Shift Registers (35 min)

Say this to the class:

“Every project so far has controlled things happening in one place at one time. Real production lines move parts through multiple stations — a part that's at Station 2 right now was at Station 1 a moment ago, and will be at Station 3 next. A shift register is how a PLC keeps track of something moving through a sequence of positions, without needing a separate sensor to identify it at every single station.”

1.1 — What a shift register actually does

A shift register is a row of bit positions — think of it as a row of boxes, each holding a 0 or a 1. On every trigger pulse, every value in the row moves one position over, a new bit is loaded in at one end, and whatever was sitting in the box at the far end falls off and is discarded. Nothing is “calculated” — the register simply remembers a pattern and slides it along, one step at a time.

The direction matters and must be fixed at design time: a **shift-left** register loads new data in on the right and pushes old data out the left; a **shift-right** does the opposite. Which direction you choose should match the physical direction the parts are actually travelling on the line, so the bit pattern on screen always matches what a person would see walking past the conveyor.

Worked example — a 4-station conveyor

Picture a conveyor with 4 fixed positions a part can occupy: Station 1 (entry) through Station 4 (exit). We use a 4-bit shift register, one bit per station, where a 1 means “a part is currently sitting at this station.” Two separate signals drive it: an **entry sensor** feeds the bit that gets loaded in at Station 1, and a single **advance pulse** (fired once per conveyor step, e.g. from a timer or an index sensor) tells the register when to shift everything along.

```
Start:                [0, 0, 0, 0]    (stations 1-2-3-4, all empty)

Part detected at entry sensor -> entry bit = 1
Advance pulse fires:   [1, 0, 0, 0]    (part now shown at Station 1)

No new part at entry -> entry bit = 0
Advance pulse fires:   [0, 1, 0, 0]    (part has moved to Station 2)

Advance pulse fires again: [0, 0, 1, 0]    (Station 3)
Advance pulse fires again: [0, 0, 0, 1]    (Station 4 - ready to exit)
Advance pulse fires again: [0, 0, 0, 0]    (part has left the register entirely)
```

Point out explicitly: the register never “knows” it's tracking a physical part — it only knows it's sliding bits along. The physical meaning (a part being present) is something we assign to the pattern by wiring each bit position's output to a lamp or downstream logic at that station.

Instructor tip: Ask the class what would happen if two parts entered close together, before the register had a chance to advance in between. The entry bit would be set to 1 twice while the register was still sitting at the same shift position, and the second part's arrival would simply be lost — it never gets its own slot. This is worth demonstrating live: it's the natural bridge into why the advance pulse and the entry sensor need to be sequenced correctly, not just wired in independently.

1.2 — The most common mistake: wiring order

Students very often wire the entry sensor's bit to load in *after* the shift has already happened on the same scan, rather than before it — or the reverse of what their specific software expects. The symptom is that every part appears to show up “one station too far” compared to where it physically is. Rather than pointing at the fix, ask: “when the advance pulse fires, does your software load the new bit in and then shift, or shift and then load the new bit in — and which one did you assume?” Letting students check their platform's documentation to answer this themselves is part of the point: shift register instruction order is one of the few things that genuinely differs between PLC platforms.

1.3 — Hands-on exercise

1. Build a 4-bit shift register. Wire a push-button as the entry sensor (feeding the incoming bit) and a separate push-button as the manual advance pulse, kept deliberately separate from each other for now.
2. Wire each of the 4 bit positions to its own lamp, labelled Station 1 through Station 4.
3. Press the entry button once, then press advance once. Confirm the Station 1 lamp turns on and the entry bit is not still “stuck” on.
4. Continue pressing advance alone (without pressing entry again) and watch the lit lamp move from Station 1 through to Station 4, then disappear entirely off the end.
5. Extra practice: trigger two entries with only one advance pulse between them, and observe the data-loss behaviour described in the instructor tip above, live on screen.

Checkpoint — before moving on

Ask a student to predict, without touching the software: starting from an empty register [0,0,0,0], if the entry sensor is triggered once and then the advance pulse fires three times total with no further entries, which station will the bit be sitting at? Correct answer: Station 3 — one advance moves it into Station 1, so the third advance places it at Station 3, not Station 4. This confirms students are counting from the first advance, not from zero.

2. Function Blocks vs. Functions — FB vs. FC (30 min)

Say this to the class:

“So far, every piece of logic you've built lives directly in your main program, out in the open. As programs grow — and a real batch plant might have five identical mixing tanks — copying the same block of logic five times becomes messy and error-prone. Today's section is about packaging logic so it can be reused cleanly, and about a distinction that trips a lot of people up: does the reused logic need to remember anything, or not?”

2.1 — FC (Function): no memory

A Function takes its current inputs, calculates a result, and produces an output — and that's the entire story. Call it again next scan with the exact same inputs, and you get the exact same output, every time, with zero regard for what happened on any previous scan. The scaling formula from Day 3 is a perfect example of something that belongs in an FC: raw value in, scaled value out, nothing remembered in between calls.

2.2 — FB (Function Block): has its own memory

A Function Block is different in one crucial way: it has its own private memory, called an **instance**, that persists from one scan to the next. This is exactly why a TON timer had to be introduced as a block rather than a plain calculation back on Day 2 — a timer has to remember its elapsed time across scans, and a stateless FC has nowhere to keep that number. Every timer and counter you've used so far has secretly been a Function Block the whole time, for exactly this reason.

Instructor tip: A clean test to give students for telling FC from FB: “if I called this block twice in a row with the same inputs, could the second answer ever be different from the first?” If yes, it needs memory, and it's an FB. If the answer is always guaranteed identical, it's a candidate for a plain FC.

2.3 — Why this matters for the Batch Mixing project ahead

The real payoff of FBs is reuse with independent memory: if a plant has five mixing tanks running the exact same fill-mix-drain sequence, you can write that sequence once as a Function Block and then call it five separate times — once per tank — as five separate **instances**. Each instance keeps its own elapsed time, its own current stage, its own everything, completely independently of the other four, even though they're all running the identical program logic underneath. Without FBs, this would mean copy-pasting the entire sequence five times and hoping every copy stays in sync if the logic is ever changed.

2.4 — Hands-on exercise

1. Build a simple custom Function Block called TankLevelCheck: it takes a raw analog level reading and a high-level setpoint as inputs, and outputs a single BOOL, TankFull.
2. Call this FB twice in the same program, as two separate instances (e.g. Tank_A_Check and Tank_B_Check), feeding each one a different simulated analog input.
3. Confirm each instance's TankFull output responds only to its own tank's level, completely independent of the other instance — sweep Tank A's input and confirm Tank B's output doesn't move at all.

Checkpoint — before moving on

Ask a student to classify each of the following as FC or FB, and to justify why in one sentence: (1) a block converting Celsius to Fahrenheit, (2) a block that counts how many parts have passed a sensor, (3) a block averaging two live sensor readings. Correct answers: (1) FC — no memory needed, same inputs always give the same output; (2) FB — must remember the running count between scans; (3) FC — a fresh average calculated from current values only, nothing retained.

3. PID Control (35 min)

Say this to the class:

“On Day 3, the temperature project used a hard GT/LT comparison, and the extra challenge introduced hysteresis to stop the fan chattering on and off. PID is the properly engineered version of that same problem: instead of a fan that’s simply ON or OFF, imagine a heating element whose power can be varied smoothly, and the goal is to land exactly on a setpoint and stay there — not oscillate around it, and not sit permanently a little below it either.”

3.1 — Starting from the problem, not the formula

Before naming P, I, and D, walk through what naturally goes wrong with a simple approach. Suppose heater power is set proportional to how far the temperature is below setpoint — a big gap gives full power, a small gap gives gentle power. This alone is **Proportional** control, and it has two predictable weaknesses students should be led to notice rather than told:

- As the gap shrinks near the setpoint, power shrinks too — the heater loses just enough drive to overcome real-world heat loss, and the tank settles a little *below* setpoint forever, never quite arriving. This permanent gap is called **steady-state error**.
- If proportional gain is turned up to fix that gap faster, the system tends to overshoot past setpoint, cool down, overshoot again in the other direction — settling into oscillation rather than a steady value.

3.2 — Introducing I: eliminating the leftover gap

The **Integral** term keeps a running total of the error over time — every scan that the tank sits even slightly below setpoint, that small persistent gap keeps adding up, and the integral term keeps nudging power upward until the gap is finally closed to zero. This is precisely why P alone leaves a permanent offset and P+I does not: I actively accumulates and corrects exactly the error that P alone gives up on. The trade-off to mention: too much integral action can itself cause overshoot, because the accumulated correction can keep pushing power upward even after the setpoint has already been reached, an effect commonly called **integral windup**.

3.3 — Introducing D: damping the overshoot

The **Derivative** term looks at how fast the error is currently changing, not its size. If temperature is rising quickly toward setpoint, D pulls power back down pre-emptively, before the setpoint is actually reached — acting like a brake that anticipates arrival rather than reacting after overshooting. This is the same underlying idea as the SUB-based rising/falling detection mentioned briefly on Day 3 — D is built from exactly that kind of rate-of-change calculation.

Worked narrative example — heating a mixing tank to 60°C

P only:	Rises quickly, but settles at ~57C and stays there - never reaches 60C.
P + I:	Rises, slightly overshoots to ~63C, oscillates, eventually settles at 60C.
P + I + D:	Rises smoothly, small overshoot to ~61C, settles at 60C quickly and stays put.

Read this progression aloud rung by rung rather than just showing the end state: P alone is fast but permanently short of target; adding I finally reaches the true target but at the cost of some overshoot and wobble; adding D calms that wobble down without giving up the accuracy I provided.

Instructor tip: Full PID tuning (formally, methods like Ziegler-Nichols) is out of scope for today — this section is deliberately conceptual, not a tuning course. The one practical heuristic worth giving students is the build order: start with P alone and observe the steady-state gap, then add I and observe it close (at the cost of some overshoot), then add D last specifically to calm that overshoot. Building it in that order, and watching each term's specific effect appear on screen, teaches the intuition far better than being handed three numbers to type in at once.

3.4 — Hands-on exercise

1. Using the PID function block available in the software, wire a simulated heater (a numeric output standing in for heater power) to raise a simulated tank temperature toward a 60°C setpoint.
2. Run with P only. Observe and record the steady-state gap — confirm the temperature settles below 60°C and stays there.
3. Enable I. Observe the gap close to zero, and note any overshoot or oscillation that appears while it does.
4. Enable D. Observe the overshoot from the previous step reduce, without losing the accuracy I provided.
5. Extra practice: step the setpoint suddenly from 60°C to 75°C mid-run, and watch how the three terms respond together to the sudden change, rather than a gradual one.

Checkpoint — before moving on

Ask a student directly: “if a heating tank is running with P-only control and settles at a temperature slightly below setpoint, permanently, which term would you add to close that gap, and which term would you add if the tank were instead oscillating wildly around the setpoint?” Correct answers: add I to close a persistent gap; add D to calm oscillation/overshoot. If a student reaches for D to fix the persistent gap, that's worth revisiting with the worked example above before moving on.

4. Day Project: Batch Mixing Station (110 min)

Say this to the class:

“Everything in this project is something you already built separately today, plus the analog and comparison work from Day 3 — a shift register to track the batch through its stages, a Function Block so the same sequence could later run on multiple tanks, and a PID loop to control mixing temperature. Nothing new is being introduced; what's new is combining all of it into one working sequence that runs on its own from start to finish.”

4.1 — The specification to give students

Write this on the board exactly, and don't pre-solve it for them:

1. Press Start: the Fill valve opens and Ingredient A flows in until the High Level sensor is triggered.
2. Once full, the Fill valve closes and the Mixer motor runs for a fixed 10 seconds (TON).
3. During mixing, a heater under PID control brings and holds the tank at a 60°C setpoint.
4. Once the 10-second mix timer completes, the Mixer motor stops and the Drain valve opens until the Low Level sensor confirms the tank is empty.
5. Once empty, the Drain valve closes and the whole cycle waits for Start to be pressed again — it does not restart automatically.

4.2 — The core idea to teach before they start building

This project has exactly one genuinely new idea, even though every individual piece is familiar: the batch moves through a fixed sequence of named **stages** — Idle, Filling, Mixing, Draining — and only one stage should ever be “active” at a time. This is precisely what the shift register from Section 1 is for, except here each bit position represents a stage of the batch rather than a physical station on a conveyor. Point this parallel out directly: the register doesn't care whether what's moving through it is a physical part or an abstract stage of a process — the underlying mechanism is identical.

Just like the Day 2 traffic light project, every stage that turns on is responsible for turning the previous stage off. Ask the class the same question from Day 2 again, deliberately: “for every stage that turns on here, what is responsible for turning the previous one off?” The traffic light handoff pattern and the batch sequence stage pattern are the same idea wearing different clothes.

4.3 — Full build sequence

1. Wire or simulate all field devices first: Fill valve, Drain valve, Mixer motor, heater output, High Level sensor, Low Level sensor, and Start button. Confirm each can be operated manually before any sequencing logic is added.
2. Build the Idle-to-Filling handoff: Start button (with the tank confirmed empty) opens the Fill valve and advances the stage register. Test this stage alone: the Fill valve should open and stay open.
3. Build the Filling-to-Mixing handoff: the High Level sensor closes the Fill valve, starts the Mixer motor, starts the 10-second TON, and advances the stage register. Test that the Fill valve closes exactly when the High Level sensor trips, not before and not with a delay.
4. Add the PID heater loop, running continuously once Mixing begins, targeting 60°C. Confirm it engages only during the Mixing stage and not before.

5. Build the Mixing-to-Draining handoff: the TON's completed output stops the Mixer motor, opens the Drain valve, and advances the stage register. Test that mixing runs for the full 10 seconds, not more and not less.
6. Build the Draining-to-Idle handoff: the Low Level sensor closes the Drain valve and returns the sequence to Idle, waiting for Start again. This is the handoff that closes the loop.
7. Run at least 2 full batch cycles from Start to Idle without any manual intervention beyond the initial Start press, confirming the cycle does not repeat on its own until Start is pressed again.

4.4 — Debugging guidance for common failures

- **Fill valve never closes.** Ask: “which exact variable is closing your Fill valve wired to — the High Level sensor, or something else left over from testing?” This is usually a stray manual override left wired in from the first isolated test.
- **Mixer and Drain valve are both active at once, briefly.** Ask: “in the rung that stops the Mixer motor, what condition is actually causing it to stop?” This is the same class of bug as Day 2's traffic light overlap — the “turn off” half of a handoff was never wired, or was wired to the wrong stage bit.
- **Heater never seems to reach 60°C.** Ask: “is the PID block's enable condition tied to the Mixing stage specifically, or is it running all the time, including during Filling and Draining when the tank level is changing?” A PID loop enabled outside the intended stage will fight against a tank that isn't in a stable state yet.
- **Sequence completes once, then never restarts even after pressing Start.** Ask: “what exact condition returns the stage register to Idle, and does pressing Start again actually satisfy that same condition, or a different one?” Usually the Idle stage was left requiring the Low Level sensor to still be active rather than genuinely waiting for a fresh Start press.

4.5 — Extra challenge, for pairs who finish early

- Package the entire Fill-Mix-Drain sequence as a single Function Block, then instantiate it twice — Tank_A and Tank_B — each with its own independent sensors and Start button, and confirm both tanks can run their own batch cycles completely independently of one another, at different points in the sequence at the same time.
- Add a second ingredient stage before mixing: Fill Ingredient A to a partial level, then Fill Ingredient B to the final High Level sensor, before mixing begins — this requires a genuine intermediate level sensor and an extra stage in the register, not just a timer guess.

End of day checkpoint

Every student/pair demonstrates the Batch Mixing Station running at least 2 full cycles from a single Start press through to returning to Idle, with the PID-controlled heater visibly reaching and holding 60°C during the Mixing stage — and can explain out loud what would happen if the Draining-to-Idle handoff rung were deleted, before leaving.